

高级数据结构第八章习题答案

学号：202533354 姓名：ZHOU MINGYUAN

例题 1

题干：

某系统中需要保存学生成绩：

学号 $\rightarrow$ 成绩

例如：

2024001 $\rightarrow$ 85

2024002 $\rightarrow$ 91

2024003 $\rightarrow$ 76

回答：

- (1) 这里的 key 是什么？
- (2) 这里的 value 是什么？
- (3) 为什么这种结构可以看成符号表？
- (4) 哈希方法主要想提高哪类操作的效率？

答案：

- (1) key 是学号。
- (2) value 是成绩。
- (3) 因为它保存的是“学号 $\rightarrow$ 成绩”的键值映射，所以可以看成符号表。
- (4) 哈希主要想提高查找、插入和删除操作的效率。

解析：

符号表的核心是按 key 找到对应 value。哈希表通过哈希函数把 key 映射到桶位置，从而在理想情况下达到接近 $O(1)$ 的访问效率。

例题 2

题干：

给定一个哈希函数：

$$h(x) = x \bmod 10$$

现有 key：

15, 28, 33, 42

回答：

- (1) $h(15)$ 等于多少？5

(2) $h(28)$ 等于多少? 8

(3) $h(33)$ 等于多少? 3

(4) $h(42)$ 等于多少? 2

(5) 哈希函数、哈希值、哈希表分别是什么意思?

答案:

(1) $h(15)=5$ 。

(2) $h(28)=8$ 。

(3) $h(33)=3$ 。

(4) $h(42)=2$ 。

(5) 哈希函数是把 key 映射到位置的函数; 哈希值是计算得到的桶号或数组下标; 哈希表是按哈希值组织和存放记录的数据结构。

解析:

除余法 $h(x)=x \bmod 10$ 取的是 x 除以 10 的余数。该余数通常作为表中的下标或 bucket 编号。

例题 3

题干:

某哈希表有 8 个 bucket, 每个 bucket 有 2 个 slot。

现在已经插入 10 个元素。

回答:

(1) 该哈希表总容量是多少?

(2) 负载因子 α 是多少?

(3) 如果再插入 4 个元素, 负载因子变为多少?

(4) 负载因子越大, 通常意味着什么?

答案:

(1) 总容量为 $8 \times 2 = 16$ 。

(2) 当前负载因子为 $10/16 = 0.625$ 。

(3) 再插入 4 个元素后, 负载因子为 $14/16 = 0.875$ 。

(4) 负载因子越大, 表越拥挤, 冲突和溢出通常越多, 查找和插入成本上升。

解析:

bucket 数量乘以每个 bucket 的 slot 数得到总容量。负载因子反映已经使用的容量比例, 是衡量哈希表拥挤程度的重要指标。

例题 4

题干:

某哈希表有 5 个 bucket, 每个 bucket 有 2 个 slot。

哈希函数为:

$$h(x)=x \bmod 5$$

依次插入：

7, 12, 17

回答：

- (1) 7 被映射到哪个 bucket? 2
- (2) 12 被映射到哪个 bucket? 2
- (3) 17 被映射到哪个 bucket? 2
- (4) 7、12、17 是否发生冲突?
- (5) 是否发生溢出?

答案：

- (1) 7 映射到 bucket 2。
- (2) 12 映射到 bucket 2。
- (3) 17 映射到 bucket 2。
- (4) 三个 key 映射到同一 bucket，发生冲突。
- (5) 发生溢出，因为 bucket 2 只有 2 个 slot，插入第三个元素 17 时放不下。

解析：

冲突指多个 key 具有相同哈希地址；溢出指目标 bucket 的可用 slot 已满。冲突不一定马上溢出，但本题三个元素会导致溢出。

例题 5

题干：

有如下两个哈希函数：

$$h1(x)=x \bmod 8$$

$$h2(x)=x \bmod 7$$

现有 key：

16, 24, 32, 40

回答：

- (1) 使用 h1 时，这些 key 分别映射到哪里? 0
- (2) 使用 h2 时，这些 key 分别映射到哪里? 2 3 4 5
- (3) 哪个哈希函数分布更均匀?
- (4) 为什么除余法中，M 通常不建议取 2 的幂?

答案：

- (1) 使用 $h1(x)=x \bmod 8$ 时，16、24、32、40 都映射到 0。
- (2) 使用 $h2(x)=x \bmod 7$ 时，分别映射到 2、3、4、5。

(3) h_2 分布更均匀。

(4) M 取 2 的幂时，结果过度依赖 key 的低位；若 key 低位规律性强，就容易集中冲突。

解析：

除余法中模数 M 的选择会直接影响分布。选择合适的素数通常能减少由输入模式导致的集中映射。

例题 6

题干：

设哈希函数为：

$h(x) = \text{字符串首字母} - 'a'$

其中 a 对应 0, b 对应 1, c 对应 2, 依此类推。

现有字符串：

apple, angle, book, cat, clock

回答：

(1) $h(\text{apple})$ 等于多少？0

(2) $h(\text{angle})$ 等于多少？0

(3) $h(\text{book})$ 等于多少？1

(4) $h(\text{cat})$ 和 $h(\text{clock})$ 分别等于多少？2 2

(5) 哪些字符串是同义词？

(6) 这个哈希函数有什么缺点？

答案：

(1) $h(\text{apple})=0$ 。

(2) $h(\text{angle})=0$ 。

(3) $h(\text{book})=1$ 。

(4) $h(\text{cat})=2$, $h(\text{clock})=2$ 。

(5) 同义词为 apple 与 angle, cat 与 clock。

(6) 缺点是只看首字母，忽略后续字符，容易产生大量冲突。

解析：

同义词是指不同 key 经过哈希函数后得到同一哈希值。好的字符串哈希函数应综合多个字符信息，而不是只使用首字母。

例题 7

题干：

哈希表大小为 7，每个 bucket 只有 1 个 slot。

哈希函数为：

$$h(x)=x \bmod 7$$

采用线性探测法，依次插入：

10, 17, 24, 8

回答：

(1) 10 插入到哪里？3

(2) 17 插入到哪里？4

(3) 24 插入到哪里？5

(4) 8 插入到哪里？1

(5) 插入完成后画出哈希表。

0

1 8

2

3 10

4 17

5 24

6

答案：

(1) 10 插入到下标 3。

(2) 17 原始地址为 3，冲突后插入到下标 4。

(3) 24 原始地址为 3，依次检查 3、4 后插入到下标 5。

(4) 8 插入到下标 1。

(5) 最终表：0:空，1:8，2:空，3:10，4:17，5:24，6:空。

解析：

线性探测在发生冲突后按 $(h(x)+1)$ 、 $(h(x)+2)$ 等顺序继续检查，越过表尾时通过取模回到表头。

例题 8

题干：

哈希表大小为 7，采用线性探测法。

当前哈希表如下：

```
-----  
index 0 1 2 3 4 5 6  
-----
```

```
Value 8 10 17 24  
-----
```

哈希函数为：

$$h(x)=x \bmod 7$$

回答：

- (1) 查找 24 时，从哪个位置开始？
- (2) 查找 24 需要依次检查哪些位置？
- (3) 查找 31 时，从哪个位置开始？
- (4) 如果 31 不在表中，什么时候可以判定查找失败？
- (5) 为什么查找不能只看 $h(x)$ 对应的位置？

答案：

- (1) 查找 24 从 $h(24)=3$ 开始。
- (2) 依次检查下标 3、4、5，在 5 找到 24。
- (3) 查找 31 从 $h(31)=3$ 开始。
- (4) 若 31 不在表中，检查 3、4、5 后到达真正空位 6 时可判定失败。
- (5) 因为开放式寻址插入时可能把元素放到后续探测位置，查找必须沿同一探测路径进行。

解析：

线性探测的查找路径必须与插入路径一致。遇到真正空位说明该 key 若曾插入就一定会在此前出现，因此可停止。

例题 9

题干：

哈希表大小为 10，每个bucket的slot数量为1。采用线性探测法。

若连续插入的一批元素最终占据了下标 2,3,4,5,6,7。

回答：

- (1) 这段连续占用区域叫什么？
- (2) 如果一个新 key 的哈希地址是 4，它会怎样插入？
- (3) 为什么这个区域会越来越长？
- (4) 线性探测法的主要缺点是什么？

答案：

- (1) 这段连续占用区域称为主集群 (primary clustering)。
- (2) 哈希地址为 4 的新 key 会依次检查 4、5、6、7，若它们已占用，则继续插入到后面的空位。
- (3) 因为落入集群内部或附近的新 key 都容易被追加到集群尾部，使连续区域变长。
- (4) 线性探测的主要缺点是容易形成主集群，增加查找和插入的探测长度。

解析：

主集群会吸引更多冲突元素，使原本局部的拥挤区域不断扩大，这是线性探测性能下降的典型原因。

例题 10

题干：

哈希表大小为 11，哈希函数为：

$$h(x) = x \bmod 11$$

随机探测序列为：

3, 5, 1, 7

现在插入 key=22。

已知位置 0 和 3 已经被占用。

回答：

- (1) key=22 的原始哈希地址是多少？0
- (2) 第一次冲突后检查哪个位置？3
- (3) 如果该位置也被占用，下一步检查哪个位置？5
- (4) 随机探测法为什么必须在插入和查找时使用同一组随机序列？

答案：

- (1) key=22 的原始哈希地址为 0。
- (2) 第一次冲突后检查 $(0+3) \bmod 11 = 3$ 。
- (3) 若 3 也被占用，下一步检查 $(0+5) \bmod 11 = 5$ 。
- (4) 插入和查找必须使用同一组随机探测序列，否则查找时可能沿不同路径而找不到已插入元素。

解析：

随机探测并不是每次真正随机生成路径，而是使用固定的伪随机偏移序列，以保证同一 key 的查找路径可重复。

例题 11

题干：

哈希表大小为 13，哈希函数为：

$$h(x) = x \bmod 13$$

采用平方探测法，探测顺序为：

$$h(x), h(x)+1^2, h(x)+2^2, h(x)+3^2 \dots$$

现插入 key=26。

已知位置 0、1、4 已经被占用。

回答：

- (1) key=26 的原始哈希地址是多少？
- (2) 第一次冲突后检查哪个位置？
- (3) 第二次冲突后检查哪个位置？
- (4) 第三次冲突后检查哪个位置？
- (5) 平方探测相比线性探测有什么优点？

答案：

- (1) key=26 的原始哈希地址为 0。
- (2) 第一次冲突后检查 $(0+1^2) \bmod 13 = 1$ 。
- (3) 第二次冲突后检查 $(0+2^2) \bmod 13 = 4$ 。
- (4) 第三次冲突后检查 $(0+3^2) \bmod 13 = 9$ 。
- (5) 平方探测跳跃式检查位置，比线性探测更能减轻主集群问题。

解析：

平方探测的探测距离为 1、4、9 等平方数，不会简单地沿连续位置前进，因此连续拥挤区不容易像线性探测那样扩张。

例题 12

题干：

哈希表大小为 7，采用线性探测法。

哈希函数为：

$$h(x)=x \bmod 7$$

当前表中：

index 0 1 2 3 4 5 6

Value 8 15 22

现在删除 15。

回答：

- (1) 能不能直接把 15 所在位置改成普通空位？
- (2) 如果直接清空，查找 22 时可能出现什么问题？
- (3) 正确做法应该放置什么标记？

(4) 查找时遇到该标记应该停止还是继续？

(5) 插入时遇到该标记能不能复用？

答案：

(1) 不能直接把 15 所在位置改成普通空位。

(2) 如果直接清空，查找 22 时可能在该空位提前停止，误判 22 不存在。

(3) 正确做法是放置 tombstone（墓碑）标记。

(4) 查找时遇到墓碑应继续探测。

(5) 插入时遇到墓碑可以复用该位置。

解析：

墓碑标记用于维持探测链不断裂。它对查找来说不是终止信号，但对插入来说可以作为可复用空间。

例题 13

题干：

哈希表大小为 5，哈希函数为：

$$h(x) = x \bmod 5$$

采用链地址法，依次插入：

3, 8, 13, 6, 11

回答：

- (1) 每个元素分别映射到哪个 bucket？
- (2) 画出链地址法下的哈希表结构。
- (3) 查找 13 时，先进入哪个 bucket？
- (4) 链地址法如何处理冲突？
- (5) 链地址法可能有什么缺点？

答案：

- (1) 3→3, 8→3, 13→3, 6→1, 11→1。
- (2) 链地址表：0: 空；1: 6→11；2: 空；3: 3→8→13；4: 空。
- (3) 查找 13 时先进入 bucket 3。
- (4) 链地址法把映射到同一 bucket 的元素接在该 bucket 后面的链表中。
- (5) 缺点是链表过长会降低查找效率，并带来额外指针或存储开销。

解析：

链地址法不在表内继续探测其他空位，而是在同一个 bucket 下保存所有同义词。哈希函数越差，链越容易变长。

例题 14

题干：

某哈希表发生冲突。

回答：

- (1) 开放式寻址如何处理冲突？
- (2) 链地址法如何处理冲突？
- (3) 哪种方法会在表内继续寻找空位？
- (4) 哪种方法会在 bucket 后维护链表？
- (5) 如果哈希函数很差，两种方法都会出现什么问题？

答案：

- (1) 开放式寻址在表数组内部按探测序列寻找其他空位。
- (2) 链地址法在 bucket 后维护链表来保存冲突元素。

- (3) 开放式寻址会在表内继续寻找空位。
- (4) 链地址法会在 bucket 后维护链表。
- (5) 如果哈希函数很差，两种方法都会性能下降：开放式寻址会有长探测序列，链地址法会有长链表。

解析：

冲突处理方法只能缓解冲突，不能替代好的哈希函数。分布不均会使任何哈希表结构局部拥挤。

例题 15

题干：

某系统最开始只存 100 条记录，后来增长到 100000 条记录。

如果仍然使用最初设计的小型静态哈希表。

回答：

- (1) 哈希表负载因子会怎样变化？
- (2) 冲突和溢出会怎样变化？
- (3) 查找效率会怎样变化？
- (4) 为什么需要动态哈希？
- (5) 动态哈希主要解决什么问题？

答案：

- (1) 负载因子会不断升高。
- (2) 冲突和溢出会越来越多。
- (3) 查找效率会下降，插入也会更慢。
- (4) 需要动态哈希来随数据增长逐步扩容。
- (5) 动态哈希主要解决静态哈希表规模固定、数据增长后性能恶化的问题。

解析：

静态哈希表一旦容量设计过小，增长后的数据会使 bucket 过度拥挤。动态哈希通过分裂和扩容保持较低负载。

例题 16

题干：

某可扩展哈希当前目录深度为 1，只看二进制最后 1 位：

0

1

每个 bucket 最多存 2 条记录。

现在有记录：

A: $f(A)=1010$

B: $f(B)=0110$

C: $f(C)=1100$

回答：

- (1) A、B、C 按最后 1 位分别进入哪个目录项？
- (2) 插入 C 时是否可能发生溢出？
- (3) 如果发生溢出，可扩展哈希通常如何处理？
- (4) 目录深度从 1 变为 2 后，目录项有哪些？
- (5) 为什么目录翻倍比重建整个文件代价低？

答案：

- (1) A、B、C 的二进制末位都是 0，因此都进入目录项 0。
- (2) 插入 C 时可能发生溢出，因为该 bucket 最多存 2 条记录。
- (3) 可扩展哈希通常分裂溢出的 bucket，必要时将目录翻倍。
- (4) 目录深度变为 2 后，目录项为 00、01、10、11。
- (5) 目录翻倍主要复制和调整目录指针，并只分裂相关 bucket，比重建整个文件代价低。

解析：

可扩展哈希用全局深度决定目录使用多少位，用局部深度决定某个 bucket 的区分位数。溢出时优先局部分裂。

例题 17

题干：

某可扩展哈希目录为：

00 → Bucket A

01 → Bucket B

10 → Bucket A

11 → Bucket B

回答：

- (1) 目录项和 bucket 是否一定一一对应？
- (2) 哪些目录项共享同一个 bucket？
- (3) 如果 Bucket B 满了，是否一定要重建所有 bucket？
- (4) 可扩展哈希中“目录”和“数据桶”的作用分别是什么？

答案：

- (1) 目录项和 bucket 不一定一一对应。
- (2) 00 和 10 共享 Bucket A；01 和 11 共享 Bucket B。
- (3) Bucket B 满了不一定要重建所有 bucket，只需分裂相关 bucket，并在必要时调整目录。
- (4) 目录用于根据哈希前缀定位 bucket；数据 bucket 用于存储真实记录。

解析：

多个目录项可以指向同一个 bucket，这是可扩展哈希节省空间和支持局部分裂的关键。

例题 18

题干：

线性哈希初始有 4 个 bucket：

0, 1, 2, 3

初始哈希函数为：

$$h_0(x) = x \bmod 4$$

每个 bucket 最多存 3 条记录。

现在插入 key=17，且 $h_0(17)=1$ 。

如果 bucket 1 已满，发生溢出。

回答：

- (1) 在线性哈希中，是否一定分裂 bucket 1？
- (2) 真正分裂哪个 bucket 由什么决定？
- (3) 如果当前分裂指针 $sp=0$ ，这次分裂哪个 bucket？
- (4) 线性哈希为什么不是谁满了就分谁？
- (5) 溢出的记录是否会直接插入失败？

答案：

- (1) 在线性哈希中，不一定分裂 bucket 1。
- (2) 真正分裂哪个 bucket 由分裂指针 sp 决定。
- (3) 若 $sp=0$ ，这次分裂 bucket 0。
- (4) 线性哈希按固定顺序渐进分裂，保证扩容平滑，而不是谁满就立即分谁。
- (5) 溢出的记录不会直接插入失败，通常会暂时放入溢出桶或溢出链。

解析：

线性哈希的核心是用分裂指针逐步推进扩容。即使某个 bucket 溢出，也可能先用溢出区保存记录，等指针轮到该 bucket 再重分布。

例题 19

题干：

线性哈希中：

$$N=4$$

$$h0(x)=x \bmod 4$$

$$h1(x)=x \bmod 8$$

当前分裂指针：

$$sp=2$$

回答：

- (1) 查找 key=8 时，先算 $h0(8)$ 等于多少？
- (2) $h0(8)$ 是否小于 sp ？
- (3) 应该用 $h0$ 还是 $h1$ 定位？
- (4) 查找 key=7 时， $h0(7)$ 等于多少？
- (5) $h0(7)$ 是否小于 sp ？
- (6) key=7 应该从哪个 bucket 开始查找？

答案：

- (1) $h0(8)=8 \bmod 4 = 0$ 。
- (2) $0 < sp=2$ ，因此小于 sp 。
- (3) 应使用 $h1$ 定位， $h1(8)=8 \bmod 8 = 0$ 。
- (4) $h0(7)=7 \bmod 4 = 3$ 。
- (5) 3 不小于 $sp=2$ 。
- (6) key=7 应从 bucket 3 开始查找。

解析：

线性哈希中，小于 sp 的 bucket 已经被分裂过，因此查找这些区域的 key 要使用下一层哈希函数 $h1$ ；未分裂区域仍使用 $h0$ 。

例题 20

题干：

线性哈希中，bucket 5 已满，但当前分裂指针 $sp=1$ 。

此时又有一个 key 应该插入 bucket 5。

回答：

- (1) bucket 5 是否会被立即分裂？
- (2) 新记录会插入失败吗？

- (3) 它通常会暂时放在哪里？
- (4) 查找该记录时，只查主 bucket 够不够？
- (5) 当以后轮到 bucket 5 分裂时，主 bucket 和溢出部分中的记录会怎样处理？

答案：

- (1) bucket 5 不会因为当前溢出而立即分裂，因为 $sp=1$ 。
- (2) 新记录不会插入失败。
- (3) 它通常暂时放入 bucket 5 的溢出桶或溢出链。
- (4) 查找该记录时只查主 bucket 不够，还要检查溢出部分。
- (5) 以后轮到 bucket 5 分裂时，主 bucket 和溢出部分中的记录会一起按新的哈希函数重新分配。

解析：

溢出桶是线性哈希平滑扩容的缓冲机制。它允许插入继续进行，同时把重分布推迟到分裂指针到达时处理。

例题 21

题干：

哈希表大小为 10，每个 bucket 只有 1 个 slot。

哈希函数为：

$$h(x)=x \bmod 10$$

依次插入：

12, 22, 32, 7

分别采用：

- A. 线性探测法
- B. 链地址法

回答：

- (1) 在线性探测法下，画出最终哈希表。
- (2) 在链地址法下，画出最终哈希表。
- (3) 两种方法处理 12、22、32 冲突的方式有什么不同？
- (4) 哪种方法会改变元素实际存放的表内位置？
- (5) 哪种方法会把同义词挂在同一个 bucket 后面？

答案：

- (1) 线性探测最终表：0:空, 1:空, 2:12, 3:22, 4:32, 5:空, 6:空, 7:7, 8:空, 9:空。
- (2) 链地址法最终表：2:12->22->32, 7:7, 其余 bucket 为空。
- (3) 线性探测把冲突元素放到表内后续空位；链地址法把冲突元素挂到同一 bucket 的链表后。
- (4) 开放式寻址会改变元素实际存放的表内位置。
- (5) 链地址法会把同义词挂在同一个 bucket 后面。

解析：

12、22、32 都映射到下标 2。线性探测通过 2、3、4 形成连续探测区；链地址法则保留 bucket 2 作为入口。

例题 22

题干：

某系统需要保存：

用户名 → 用户信息

用户数量一开始较少，但未来可能快速增长。

回答：

- (1) 为什么可以使用哈希表？
- (2) 哈希函数的作用是什么？
- (3) 如果两个用户名映射到同一个 bucket，叫什么？
- (4) 如果 bucket 已满，叫什么？
- (5) 静态哈希中可以用哪些方法处理冲突？
- (6) 当用户数量不断增长时，为什么需要动态哈希？
- (7) 可扩展哈希和线性哈希在扩容方式上有什么区别？

答案：

- (1) 可以使用哈希表，因为系统需要按用户名快速定位用户信息。
- (2) 哈希函数把用户名映射到 bucket 或数组下标。
- (3) 两个用户名映射到同一个 bucket 称为冲突。
- (4) bucket 已满称为溢出。
- (5) 静态哈希可用开放式寻址、链地址法或溢出桶等方式处理冲突。
- (6) 用户数量增长会提高负载因子并增加冲突，因此需要动态哈希扩容。
- (7) 可扩展哈希按溢出的 bucket 局部分裂，必要时目录翻倍；线性哈希按分裂指针逐个 bucket 平滑分裂。

解析：

本题把符号表、哈希函数、冲突处理和动态扩容串联起来。核心思想是：哈希表适合 key-value 快速访问，但必须控制冲突和负载因子。